Text Representation

1) OHE
2) BOW
3) N-grams
4) TF-IDF

One Hot Encoding

4 elements

0 [1, 0, 0, 0]
1 [0, 1, 0, 0]
2 [0, 0, 1, 0]
3 [0, 0, 0, 1]

Binary Vectors

Vector length
⇓
Vocabulary Size

	cat	dog	fish	bird
Cat →	1	0	0	0
Dog →	0	1	0	0
fish →	0	0	1	0
bird →	0	0	0	1

Advantages

Simple & Intuitive

Easy to Implement

No Info loss

Disadvantages

Huge Vocabulary Size
* high dimensional *

Extremely Sparse vectors

No Semantic meaning

* All words are equi valent distant.

Can't handle out of vocab words

2) Bag of Words (BOW)

Text Document Representation

Sen 1 :- John likes to watch movies. Mary likes too.

Sen 2 :- John likes to watch football.

Dict

{ 1 : John , 2 : likes , 3 : to , 4 : watch , 5 : movies

6 : football , 7 : Mary , 8 : too }

Text Representation (No of Appearances)

{ John : 2, likes : 3, to : 2, watch : 2, movies : 1, football : 1, Mary : 1, too : 1 }

Vectors (Index)

[1, 2, 3, 4, 5, 7, 2, 8] X

[1, 2, 3, 4, 6] X

Vocabulary Size = 8

[2, 3, 2, 2, 1, 1, 3, 1]

[2, 3, 2, 2, 2, 1, 0, 0, 0]

Vectors (Based on no of appearances)

Ignoring grammar OR order

Vocabulary

[cat, dog, bird] {1x3}

Doc 1: cat dog cat → [2, 1, 0]

Doc 2: dog bird → [0, 1, 1]

Doc 3: cat cat cat cat → [4, 0, 0]

Dimension :- 3

Advantages

1) Simple & Intuitive
2) Captures Word Frequency
3) Can handle vocab size

Disadvantages

1) Losses order
2) Sparse Vectors
3) Common words dominate more.

{ Dog bite the man → Doc 1
  Man bite the dog → Doc 2 }

Vocab :- [Dog, bite, the, man]

[1, 1, 1, 1]

[1, 1, 1, 1]

Dictionary :- 1 million words

Sen :- 10 words

Vocab Size :- $[1,000,000]^{*}_{-10}$

[

N-GRAMS

GRAM :- A word

N - (number i.e, 1, 2, 3, 4 5 ... etc)

1 - GRAM :- uniqram (individual words)

2 - GRAM :- biqram (pairs of consecutive words)

3 - GRAM :- triqram (Triplets of consecutive words)

4 - GRAM :- quadgrams

I love NLP

Uniqram = ["I", "love", "NLP"]

Biqram = ["I love", "love NLP"]

Triqram = ["I love NLP"]

{love, I}

Advantages

1) Word Order ([I, love])

2) Context is captured.

3)

I, love, NLP

New York York New

Disadvantages

1) Sparse Vectors
2) Very high Memory Requirements

TF - IDF

Term Frequency - Inverse Document Frequency

$$TF = \frac{\text{No of times Term(t) appears in the document}}{\text{Total no of terms in the document}}$$

* Cat appears 1 time in the document *
* Total words in the document = 6

$$TF = 1/6 \approx 0.167$$

$$TF = 3/6 = 0.5$$

$$TF = 6/6 = 1$$

TF :- How often a term appears in a doc.

IDF :- Inverse Document Frequency

Book :- 1 Page = 1 Doc

↓

100 pages = 100 Documents

How common a term appears across all the docs.

$$IDF(t, D) = \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$

N = Total no of docs

Denominator :- No of docs containing the term t

IDF

1) Total Docs, $N = 3$

D1 = 1

D2 = 0

D3 = 1

$IDF = \log\left(\frac{3}{2}\right)$

≈ 0.1

What will be the TF · IDF

$TF \approx 0.167$

$IDF \approx 0.176$

$TF - IDF = 0.167 \times 0.176$

≈ 0.029

TF IDF Relationship

Word Type	TF	IDF	TF-IDF
Common / Stop	High	Low	Low
Rare but relevant	Med/Mod	High	high
Rare but irrelevant	Low	High	low